

数据结构与算法基础

第一章 线性结构

数组支持随机访问，按下标读取是 $O(1)$ ，但插入和删除需要移动元素，平均是 $O(n)$ 。

链表通过指针连接节点，插入删除只需修改指针，是 $O(1)$ ，但访问第 n 个元素必须从头遍历。

栈是后进先出的结构，支持函数调用栈、括号匹配、表达式求值等场景；队列是先进先出的结构，广泛用于任务调度和消息缓冲。

第二章 树结构

二叉搜索树左子树小于根、右子树大于根，查找平均 $O(\log n)$ ，退化成链表时变成 $O(n)$ 。

AVL 树通过严格平衡保证高度，红黑树用近似平衡换取更少的调整次数，是 Java HashMap 和 TreeMap 的底层结构之一。

B+ 树是多路平衡搜索树，树高更低，一个节点对应磁盘一页，能显著减少磁盘 I/O，是 MySQL 索引的标准实现。

堆是完全二叉树，可以在 $O(1)$ 取最值、 $O(\log n)$ 插入和调整，是优先队列的底层实现。

第三章 排序算法

冒泡、插入、选择排序的时间复杂度是 $O(n^2)$ ，适合小规模或基本有序的数据。

归并排序稳定且最坏情况仍是 $O(n \log n)$ ，但需要额外空间；快速排序平均性能最好，最坏退化为 $O(n^2)$ ，随机选取基准可以降低退化概率。

堆排序空间复杂度为 $O(1)$ 但不稳定；计数排序和桶排序在数据范围有限的场景可以达到线性时间。

稳定排序保证相等元素的相对顺序不变，多字段排序时非常关键。

第四章 复杂度分析

大 O 表示法描述算法耗时随规模增长的趋势，忽略常数和低阶项。

常见复杂度从低到高： $O(1)$ 、 $O(\log n)$ 、 $O(n)$ 、 $O(n \log n)$ 、 $O(n^2)$ 、 $O(2^n)$ 。二分查找是 $O(\log n)$ ，哈希表平均查找是 $O(1)$ 。

分析算法时要关注最坏、平均和最好情况的差异，同时权衡时间与空间，例如用哈希表换空间把两层循环降为一次遍历。